

Scala 3 for Beginners

A complete 3-hour instructor-led course

FROM FIRST EXPRESSION TO A SMALL CONSOLE APPLICATION

Level	Duration	Language	Format
Beginner	180 minutes	English	Theory + live coding + exercises

COURSE PROMISE By the end of this session, learners will understand Scala's core syntax, model data with classes and case classes, use collections and pattern matching, and build a small working application.

[Instructor guide](#) • [Learner notes](#) • [Exercises](#) • [Answer key](#)

How to Use This Course

This document is designed for a live, three-hour beginner session. It can also be used for self-study. The instructor should type most examples live, ask learners to predict results before execution, and allow short pauses for experimentation.

Learning outcomes

- Explain what Scala is and where it is used.
- Run Scala code and recognize expressions, values, variables, and types.
- Write conditional logic, loops, and reusable methods.
- Distinguish class, object, case class, trait, and case.
- Work with List, Vector, Set, Map, Option, and common collection operations.
- Use pattern matching to handle structured data safely.
- Build and explain a small console-based inventory application.

Prerequisites and setup

- No prior Scala experience is required.
- Basic programming knowledge is helpful but not mandatory.
- Recommended: Scala 3 installed through Scala CLI, or an online Scala 3 playground.
- Verify the environment with: scala-cli version

```
// Save as Hello.scala, then run: scala-cli Hello.scala
@main def hello(): Unit =
  println("Hello, Scala!")
```

Teaching principles

Principle	Instructor action
Predict first	Ask learners what the program will print before running it.
Small steps	Change one element at a time and rerun.
Prefer immutability	Introduce val before var and immutable collections before mutable ones.
Use the REPL	Test expressions quickly and make errors visible and normal.
Check understanding	Use the checkpoint questions at the end of each module.

180-Minute Course Plan

Time	Duration	Module	Method
00:00-00:10	10 min	Welcome, goals, and first program	Demo + discussion
00:10-00:35	25 min	1. Values, types, expressions, and strings	Teach + practice
00:35-01:05	30 min	2. Control flow and methods	Teach + coding
01:05-01:15	10 min	Exercise 1: Price calculator	Individual/pairs
01:15-01:25	10 min	Break	Pause
01:25-02:00	35 min	3. Classes, objects, case classes, and traits	Teach + modeling
02:00-02:30	30 min	4. Collections, Option, and transformations	Teach + practice
02:30-02:45	15 min	5. Pattern matching and error handling	Teach + challenge
02:45-02:57	12 min	Capstone: Mini inventory application	Guided build
02:57-03:00	3 min	Review and next steps	Exit ticket

TIMING NOTE The schedule includes 170 minutes of instruction and practice plus a 10-minute break, for exactly 180 minutes.

Suggested diagnostic opening questions

1. Have you programmed before? If yes, in which language?
2. What do you expect from a language that combines object-oriented and functional programming?
3. What is the difference between a value and a variable?

Opening - Meet Scala

What is Scala?

Scala is a statically typed programming language that runs primarily on the Java Virtual Machine (JVM). Its name comes from “scalable language.” It combines object-oriented programming, where behavior is organized around objects, with functional programming, where functions and immutable values play a central role.

Characteristic	Meaning for a beginner
Statically typed	Many mistakes are detected before the program runs.
Type inference	Scala often knows the type without requiring you to write it.
JVM ecosystem	Scala can use Java libraries and tools.
Expression-oriented	Constructs such as if can produce values.
Multi-paradigm	Object-oriented and functional styles can coexist.

Your first program

```
@main def hello(): Unit =  
  val learner = "Ada"  
  println(s"Hello, $learner! Welcome to Scala.")
```

Read the code aloud: @main marks the entry point; hello is the method name; Unit means the method is used for its effect rather than for a useful returned value; val creates an immutable binding; println displays text.

ASK Which parts of this program look familiar from another language? Which parts are new?

REPL versus source file

- The REPL evaluates one expression at a time and is excellent for exploration.
- A .scala source file stores a complete program that can be compiled and rerun.
- In both cases, compiler messages are part of the learning process: read the first error carefully.

Module 1 - Values, Types, and Expressions

1.1 Values with val and variables with var

```
val productName = "Tiger Roots" // cannot be reassigned
var stock = 12 // can be reassigned
stock = stock - 1

println(productName)
println(stock)
```

Use val by default. A val makes code easier to reason about because the binding cannot point to a different value. Use var only when the changing state is intentional and clear.

COMMON ERROR productName = "Other" does not compile because productName was declared with val.

1.2 Common types

Type	Example	Purpose
Int	42	Whole numbers
Long	30000000000L	Large whole numbers
Double	12.5	Decimal numbers
Boolean	true	Logical truth values
Char	'S'	One character
String	"Scala"	Text
Unit	()	No meaningful result; often used for side effects

```
val quantity: Int = 3
val unitPrice: Double = 1200.0
val available: Boolean = true
val total = quantity * unitPrice // inferred as Double
```

1.3 Expressions produce values

```
val age = 20
val category = if age >= 18 then "adult" else "minor"
println(category)
```

In Scala, if is an expression: it returns a result. This is different from languages where if is only a statement.

1.4 Operators and strings

```
val a = 10
val b = 3
println(a + b)    // 13
println(a / b)    // 3: integer division
println(a % b)    // 1: remainder
println(a > b)    // true
println(a == b)   // false
println(a > 0 && b > 0) // true
```

String interpolation

```
val name = "Scala Book"
val price = 29.9
println(s"$name costs €$price")
println(f"$name costs €$price%.2f")
```

The `s` prefix inserts variables and expressions into a string. The `f` prefix also supports numeric formatting.

A block is also an expression

```
val finalPrice =
  val basePrice = 100.0
  val taxRate = 0.20
  basePrice * (1 + taxRate)

println(finalPrice) // 120.0
```

The last expression of a block becomes the value of that block. No explicit return is needed.

Exercise - Quick type predictions (5 minutes)

GOAL Predict each result and type before using the REPL.

1. Evaluate `5 / 2` and `5.0 / 2`. Explain the difference.
2. Create `val city = "Lomé"` and print: `Welcome to Lomé!` using interpolation.
3. Create a Boolean named `affordable` that is true when price is at most 1000.
4. Create a block that calculates a subtotal and then adds 18% tax.

Checkpoint

- Why should `val` normally be preferred to `var`?
- What does type inference do?
- Why can an `if` expression be assigned to a `val`?
- What is the result of integer division `7 / 2`?

Module 2 - Control Flow and Methods

2.1 Conditional logic

```
def shippingFee(orderTotal: Double): Double =  
  if orderTotal >= 50 then 0.0  
  else if orderTotal >= 25 then 4.0  
  else 7.0
```

Scala 3 uses then after the condition. Indentation defines the block, so consistent spacing matters.

2.2 while loops

```
var count = 3  
while count > 0 do  
  println(count)  
  count -= 1  
println("Go!")
```

A while loop is useful when repetition depends on changing state. In application code, collection transformations are often clearer when processing data.

2.3 for loops and ranges

```
for number <- 1 to 5 do  
  println(number)  
  
for number <- 1 until 5 do  
  println(number) // 1, 2, 3, 4
```

NOTICE to includes the upper bound; until excludes it.

For expressions

```
val squares = for number <- 1 to 5 yield number * number  
println(squares) // Vector(1, 4, 9, 16, 25)
```

2.4 Defining methods

```
def calculateTotal(quantity: Int, unitPrice: Double): Double =  
  quantity * unitPrice  
  
val total = calculateTotal(3, 12.5)  
println(total)
```

Part	Example	Meaning
Name	calculateTotal	How callers refer to the method
Parameters	quantity: Int	Inputs and their types
Return type	Double	Type of the produced result
Body	quantity * unitPrice	Last expression is returned

Methods with multiple branches

```
def describeTemperature(celsius: Double): String =  
  if celsius < 10 then "cold"  
  else if celsius < 25 then "mild"  
  else "hot"
```

Methods returning Unit

```
def printReceipt(item: String, total: Double): Unit =  
  println("--- RECEIPT ---")  
  println(f"$item: €$total%.2f")
```

2.5 Functions as values - a first look

```
val double: Int => Int = number => number * 2  
println(double(6)) // 12  
  
val addTax = (price: Double) => price * 1.20
```

A method is defined with `def`. A function value can be stored in a `val` and passed to other methods. This becomes especially useful with collections.

BEGINNER RULE Use `def` for named reusable behavior. Recognize function values now; you will use them with `map` and `filter` later.

Exercise 1 - Price Calculator

SCENARIO A shop needs a function that calculates the price of an order after a quantity discount.

Requirements

1. Write a method `orderTotal(quantity: Int, unitPrice: Double): Double`.
2. Apply a 10% discount when quantity is 10 or more; otherwise apply no discount.
3. Create values for `quantity = 12` and `unitPrice = 600.0`.
4. Print a sentence showing the quantity and the final total with two decimal places.
5. Bonus: reject a negative quantity by returning 0.0.

Starter code

```
def orderTotal(quantity: Int, unitPrice: Double): Double =  
  // Your code here  
  ???  
  
@main def runCalculator(): Unit =  
  val quantity = 12  
  val unitPrice = 600.0  
  val total = orderTotal(quantity, unitPrice)  
  // Print the result
```

Instructor prompts

- What should the method return when quantity is 9?
- Where should the discount rate be stored?
- Can the whole if expression be assigned to a `val`?
- What happens if quantity is zero?

STOP POINT Do not show the answer yet. The complete solution is in the answer key.

Break

1. What is the difference between `val` and `var`?
2. What is the last expression of a method body used for?
3. When does `1 to 5` differ from `1 until 5`?
4. How would you explain the return type `Unit`?

Module 3 - Modeling with Scala

3.1 Regular classes

```
class Product(val name: String, var stock: Int):  
  def sell(quantity: Int): Unit =  
    if quantity > 0 && quantity <= stock then  
      stock -= quantity  
  
  def description: String =  
    s"$name - $stock units available"  
  
val drink = new Product("Tiger Roots", 12)  
drink.sell(2)  
println(drink.description)
```

A class is a blueprint. Each use of new creates a separate instance with its own state. Constructor parameters prefixed by val or var become accessible fields.

3.2 Objects and singletons

```
object PriceRules:  
  val taxRate = 0.18  
  
  def withTax(price: Double): Double =  
    price * (1 + taxRate)  
  
println(PriceRules.withTax(1000))
```

An object has exactly one instance and is accessed directly without new. It is useful for shared constants and stateless utility methods.

Companion objects

```
class Customer(val name: String, val id: Int)  
  
object Customer:  
  private var nextId = 1  
  
  def create(name: String): Customer =  
    val customer = new Customer(name, nextId)  
    nextId += 1  
    customer  
  
val ada = Customer.create("Ada")
```

A class and object with the same name in the same file are companions. The object commonly provides factory methods related to the class.

3.3 Case classes: data with useful features

```
case class OrderLine(product: String, quantity: Int, unitPrice: Double):  
  def total: Double = quantity * unitPrice  
  
val first = OrderLine("Scala Book", 2, 25.0)  
val same = OrderLine("Scala Book", 2, 25.0)  
val updated = first.copy(quantity = 3)  
  
println(first)           // readable representation  
println(first == same)   // true: structural equality  
println(updated.total)   // 75.0
```

Feature	Regular class	Case class
Construction	Usually new Product(...)	OrderLine(...)
Equality by content	Not automatic	Automatic
Readable toString	Not automatic	Automatic
copy method	Not automatic	Automatic
Pattern matching	Possible with extra work	Designed for it

USE CASE Case classes are the usual choice for immutable domain data such as customers, products, orders, and messages.

3.4 Traits: shared contracts and behavior

```
trait Priced:  
  def price: Double  
  
  def formattedPrice: String = f"€$price%.2f"  
  
case class Book(title: String, price: Double) extends Priced  
case class Drink(name: String, price: Double) extends Priced  
  
val book = Book("Learning Scala", 29.9)  
println(book.formattedPrice)
```

A trait defines capabilities that different classes can share. It can contain abstract members and implemented methods. A class can extend one class and mix in multiple traits.

3.5 case, case object, and enum

case in pattern matching

```
val number = 2
val word = number match
  case 1 => "one"
  case 2 => "two"
  case _ => "another number"
```

Each case describes a pattern. The underscore is the wildcard pattern: it matches anything not handled earlier.

case object for one meaningful value

```
sealed trait OrderStatus
case object Pending extends OrderStatus
case object Shipped extends OrderStatus
case object Cancelled extends OrderStatus
```

Scala 3 enum: a compact closed model

```
enum OrderStatus:
  case Pending
  case Shipped(trackingCode: String)
  case Cancelled(reason: String)
```

Use a case object when a case contains no data. Use a case class when each instance carries data. In Scala 3, enum provides a concise way to group a closed set of alternatives.

Concept selection guide

Construct	Choose it when...	Example
class	You need multiple instances, often with behavior or mutable state.	class Counter(var value: Int)
object	You need one shared singleton.	object Configuration
case class	You model immutable data.	case class User(name: String)
trait	Several types share a contract or behavior.	trait Printable
case object	One data-free alternative has a name.	case object Pending
case	You describe one pattern inside match.	case 0 => "zero"
enum	You need a closed family of alternatives.	enum Direction: case North, South

Modeling exercise

SCENARIO Model a simple library. A book has an ISBN, title, and availability. A shared catalog can search books by ISBN.

1. Create a case class `Book(isbn: String, title: String, available: Boolean)`.
2. Create two `Book` values.
3. Use `copy` to produce a borrowed version of the first book.
4. Create an object `Catalog` containing a method `isAvailable(book: Book): Boolean`.
5. Stretch: define a trait `Displayable` with `def display: String` and make `Book` extend it.

```
// Expected style - complete the missing parts
trait Displayable:
  def display: String

case class Book(/* fields */) extends Displayable:
  def display: String = /* formatted description */

object Catalog:
  def isAvailable(book: Book): Boolean = /* result */
```

Checkpoint

- Why does a regular class often use `new` while a case class does not?
- When would object be more appropriate than class?
- Which generated case-class feature makes non-destructive updates convenient?
- What is the difference between case and case class?

Module 4 - Collections and Option

4.1 Immutable collections

Collection	Property	Example
List	Ordered sequence; efficient access at the front	List(10, 20, 30)
Vector	Ordered sequence; efficient general-purpose indexing	Vector(10, 20, 30)
Set	Unique elements	Set("red", "blue")
Map	Key-value associations	Map("A" -> 10, "B" -> 20)

```
val products = List("Book", "Pen", "Notebook")
val prices = Map("Book" -> 25.0, "Pen" -> 2.0)
val tags = Set("scala", "beginner", "jvm")

val moreProducts = products :+ "Pencil"
println(products)      // original list is unchanged
println(moreProducts)
```

4.2 map, filter, and fold

```
val prices = List(10.0, 25.0, 50.0)

val withTax = prices.map(price => price * 1.20)
val expensive = prices.filter(price => price >= 25.0)
val total = prices.foldLeft(0.0)((sum, price) => sum + price)

println(withTax)
println(expensive)
println(total)
```

Operation	Question it answers	Output shape
map	How do I transform every element?	One result per input element
filter	Which elements should I keep?	Zero or more original elements
foldLeft	How do I combine all elements?	One accumulated result
foreach	What effect should I perform for each element?	Unit

4.3 Short lambda syntax

```
val numbers = List(1, 2, 3, 4)

val doubledA = numbers.map(number => number * 2)
val doubledB = numbers.map(_ * 2)

val even = numbers.filter(_ % 2 == 0)
```

The underscore is a shorthand parameter when the meaning remains obvious. Prefer the named form when an expression becomes difficult to read.

4.4 For comprehensions with collections

```
val names = List("Ada", "Linus", "Grace")

val messages = for
  name <- names
  if name.length >= 5
yield s"Hello, $name"

println(messages)
```

4.5 Option: a value may be absent

```
val prices = Map("Book" -> 25.0, "Pen" -> 2.0)

val bookPrice: Option[Double] = prices.get("Book") // Some(25.0)
val bagPrice: Option[Double] = prices.get("Bag")    // None

println(bookPrice.getOrElse(0.0))
println(bagPrice.getOrElse(0.0))
```

Option[A] means either Some(value) or None. It makes absence explicit and reduces reliance on null.

Transforming Option safely

```
def discountedPrice(prices: Map[String, Double], name: String): Option[Double] =
  prices.get(name).map(_ * 0.90)

val message = prices.get("Book")
  .map(price => f"Price: €$price%.2f")
  .getOrElse("Product not found")
```

AVOID Do not call `.get` on an `Option` in beginner code. It throws an exception when the value is `None`.

Collection challenge

```
case class Product(name: String, price: Double, stock: Int)

val products = List(
  Product("Book", 25.0, 4),
  Product("Pen", 2.0, 0),
  Product("Notebook", 8.0, 12)
)
```

1. Use filter to keep products that are in stock.
2. Use map to obtain only their names.
3. Calculate the total value of all inventory: price × stock for every product.
4. Find the product named Pen with find. The result type is Option[Product].
5. Create a user-friendly message when a product is absent.

HINT products.find(_.name == "Pen") returns an Option. Continue with map and getOrElse.

Checkpoint

- Which collection guarantees unique elements?
- How do map and filter differ?
- What are the two possible forms of Option?
- Why is getOrElse safer than get?

Module 5 - Pattern Matching and Safe Errors

5.1 Matching values

```
def dayType(day: String): String =
  day.toLowerCase match
    case "saturday" | "sunday" => "weekend"
    case "monday" | "tuesday" | "wednesday" |
      "thursday" | "friday" => "weekday"
    case _ => "unknown day"
```

5.2 Matching case classes

```
case class Payment(method: String, amount: Double)

def describe(payment: Payment): String =
  payment match
    case Payment("cash", amount) => f"Cash payment: €$amount%.2f"
    case Payment(method, amount) if amount > 1000 =>
      s"Large $method payment"
    case Payment(method, amount) =>
      f"$method payment: €$amount%.2f"
```

Pattern matching can extract case-class fields. A guard introduced by `if` adds a condition to a case.

5.3 Matching Option

```
def showPrice(price: Option[Double]): String =
  price match
    case Some(value) => f"€$value%.2f"
    case None        => "Not available"
```

5.4 Either for success or a useful error

```
def validateQuantity(quantity: Int): Either[String, Int] =
  if quantity > 0 then Right(quantity)
  else Left("Quantity must be positive")

validateQuantity(-2) match
  case Right(value) => println(s"Accepted: $value")
  case Left(error)  => println(s"Error: $error")
```

Option models presence or absence. `Either[L, R]` can preserve an error explanation in `Left` or a successful result in `Right`.

Exercise - Status message

GOAL Use exhaustive pattern matching over an enum.

1. Define enum `StockStatus` with cases `Available(quantity: Int)`, `OutOfStock`, and `Discontinued`.
2. Write `def statusMessage(status: StockStatus): String`.
3. Return a different clear message for every case.

Capstone - Mini Inventory Application

OBJECTIVE Combine case classes, collections, Option, methods, and pattern matching in one small console application.

Requirements

1. Model a Product with id, name, price, and stock.
2. Store at least three products in a List.
3. Implement findProduct(id), returning Option[Product].
4. Implement inventoryValue, returning the total price × stock.
5. Implement sell(id, quantity), returning Either[String, Product].
6. Print one successful sale, one error, and the inventory value.

Guided starter

```
case class Product(id: Int, name: String, price: Double, stock: Int)

object Inventory:
  val products = List(
    Product(1, "Book", 25.0, 4),
    Product(2, "Pen", 2.0, 20),
    Product(3, "Notebook", 8.0, 12)
  )

  def findProduct(id: Int): Option[Product] =
    ???

  def inventoryValue: Double =
    ???

  def sell(id: Int, quantity: Int): Either[String, Product] =
    ???

@main def runInventory(): Unit =
  // Call the methods and print the results
```

Suggested build order

- Minute 1-2: read the model and data.
- Minute 3-4: implement findProduct using find.
- Minute 5-7: implement inventoryValue using map + sum or foldLeft.
- Minute 8-10: implement sell using pattern matching and copy.
- Minute 11-12: run tests and explain the types.

Review and Exit Ticket

The mental model

Need	Scala tool
A name that should not be reassigned	val
Reusable calculation	def
Several stateful instances	class
One shared instance	object
Immutable domain data	case class
Shared capability	trait
Closed alternatives	enum
Transform elements	map
Keep selected elements	filter
Possible absence	Option
Success or explained failure	Either
Branch by data shape	match / case

Exit ticket

1. Explain val versus var in one sentence.
2. Name one feature automatically supplied by a case class.
3. What does Option protect us from?
4. Write one expression using map.
5. What would you like to practise next?

Recommended next steps

- Rewrite the capstone so Inventory stores products in a Map[Int, Product].
- Learn tuples, named and default parameters, and extension methods.
- Practise immutable updates and for comprehensions.
- Create a small sbt project and add automated tests.
- Then explore Futures, effect systems, web frameworks, or data engineering according to your goals.

Answer Key

Module 1 - Quick type predictions

```
5 / 2      // 2: Int
5.0 / 2    // 2.5: Double

val city = "Lomé"
println(s"Welcome to $city!")

val price = 900
val affordable = price <= 1000

val totalWithTax =
  val subtotal = 100.0
  subtotal * 1.18
```

Exercise 1 - Price calculator

```
def orderTotal(quantity: Int, unitPrice: Double): Double =
  if quantity < 0 then 0.0
  else
    val subtotal = quantity * unitPrice
    val discountRate = if quantity >= 10 then 0.10 else 0.0
    subtotal * (1 - discountRate)

@main def runCalculator(): Unit =
  val quantity = 12
  val unitPrice = 600.0
  val total = orderTotal(quantity, unitPrice)
  println(f"$quantity items: total = $total%.2f")
```

Expected total: 6480.00. The subtotal is 7200.00 and the 10% discount is 720.00.

Module 3 - Modeling exercise

```
trait Displayable:
  def display: String

case class Book(isbn: String, title: String, available: Boolean)
  extends Displayable:
    def display: String =
      s"$title ($isbn) - available: $available"

object Catalog:
  def isAvailable(book: Book): Boolean = book.available

val first = Book("978-1", "Scala Basics", true)
val second = Book("978-2", "Functional Thinking", true)
val borrowed = first.copy(available = false)
```

Module 4 - Collection challenge

```
case class Product(name: String, price: Double, stock: Int)

val products = List(
  Product("Book", 25.0, 4),
  Product("Pen", 2.0, 0),
  Product("Notebook", 8.0, 12)
)

val inStock = products.filter(_._stock > 0)
val names = inStock.map(_._name)
val inventoryValue = products.map(p => p._price * p._stock).sum

val pen: Option[Product] = products.find(_._name == "Pen")
val bagMessage = products.find(_._name == "Bag")
  .map(p => s"Found: ${p._name}")
  .getOrElse("Product not found")
```

The inventory value is 196.0: Book 100.0 + Pen 0.0 + Notebook 96.0.

Module 5 - Status message

```
enum StockStatus:
  case Available(quantity: Int)
  case OutOfStock
  case Discontinued

def statusMessage(status: StockStatus): String =
  status match
    case StockStatus.Available(quantity) =>
      s"$quantity units available"
    case StockStatus.OutOfStock =>
      "Temporarily out of stock"
    case StockStatus.Discontinued =>
      "This product is discontinued"
```

Capstone Solution

```
case class Product(id: Int, name: String, price: Double, stock: Int)

object Inventory:
  val products = List(
    Product(1, "Book", 25.0, 4),
    Product(2, "Pen", 2.0, 20),
    Product(3, "Notebook", 8.0, 12)
  )

  def findProduct(id: Int): Option[Product] =
    products.find(_.id == id)

  def inventoryValue: Double =
    products.map(p => p.price * p.stock).sum

  def sell(id: Int, quantity: Int): Either[String, Product] =
    findProduct(id) match
      case None =>
        Left(s"Product $id not found")
      case Some(_) if quantity <= 0 =>
        Left("Quantity must be positive")
      case Some(product) if quantity > product.stock =>
        Left(s"Only ${product.stock} units are available")
      case Some(product) =>
        Right(product.copy(stock = product.stock - quantity))

@main def runInventory(): Unit =
  println(Inventory.sell(1, 2))
  println(Inventory.sell(99, 1))
  println(f"Inventory value: €${Inventory.inventoryValue}%.2f")
```

Discussion

- findProduct returns Option because an ID may not exist.
- sell returns Either because failures deserve explanations.
- copy creates an updated Product without mutating the original.
- This simple version returns the updated product but does not replace it inside products. That is a useful next exercise in immutable state management.

Instructor Reference

Common beginner misconceptions

Misconception	Correction
val makes the object deeply immutable.	val prevents reassignment of the binding; the referenced object may still contain mutable state.
Every line needs return.	The final expression is normally the result; explicit return is rarely needed.
case means the same as case class.	case introduces a pattern; case class defines a data type with generated features.
object is the same as a JavaScript object literal.	A Scala object declares a named singleton instance.
Option is a collection containing null.	Option is Some(value) or None and should be transformed safely.
map changes the original List.	Immutable collections return a new collection.
Unit means the method returns nothing at all.	Unit has one value, (), and signals that the useful purpose is usually an effect.

Fast formative assessment

1. Prediction: What does `List(1, 2, 3).filter(_ > 1).map(_ * 10)` produce?
2. Correction: Why does `val x = if true then 1 else "one"` have an overly broad inferred type?
3. Design: Would `Customer` normally be a class, object, or case class? Explain the data and behavior involved.
4. Safety: Replace `products.get("missing").get` with a safe alternative.
5. Modeling: Choose `Option` or `Either` for a login function and justify the choice.

Optional homework

Extend the inventory application with `addProduct`, `restock`, and an immutable update of the product list. Add an enum `Category` and display only products in a chosen category. Validate that price and stock are non-negative, returning `Either[String, Product]`.